

Relatório do EP de MAC0219

Felipe Pereira Ramos Barboza - 11820322 (IME-USP)

Francisco Nassif Membrive - 12553782 (IME-USP)

Guilherme Luiz Pereira de Almeida - 14576169 (IME-USP)

24 de novembro de 2024

Resumo

Este trabalho teve como objetivo comparar o desempenho de diferentes implementações para o cálculo da dispersão de calor em um cômodo com lareira. As implementações foram feitas em GPU usando CUDA e em CPU usando C. Os testes variaram tamanho do bloco de threads (na versão em CUDA) e do grid para realizar os cálculos utilizando o método das diferenças finitas. Para comparação, foram registrados os valores de speedup, razão entre o tempo em CPU e o tempo em GPU.

Índice

1	Introdução	3
2	Metodologia e Testes Realizados	3
2.1	Implementações e variações	3
2.2	Objetivos dos Testes	3
2.3	Procedimento Experimental	3
3	Resultados	5
3.1	Impacto do Tamanho da Grade (n)	6
3.2	Efeito do Tamanho do Bloco	6
3.3	Impacto da Inserção do Corpo Quente	6
4	Conclusão	7

1 Introdução

Neste trabalho, o objetivo principal foi analisar e comparar o desempenho de diferentes abordagens para a simulação da distribuição de calor em uma grade bidimensional, utilizando implementações sequenciais em C e paralelas em CUDA. O problema escolhido é altamente paralelizável e ideal para visualizar o ganho de desempenho.

O presente trabalho está alinhado com os objetivos da disciplina de Programação Concorrente e Paralela, que visa familiarizar os alunos com os conceitos e termos básicos de sistemas paralelos, além de apresentar aplicações práticas que demonstram os benefícios da paralelização.

Além disso, o trabalho inclui a análise do impacto da inserção de um corpo quente na grade, o que modifica a distribuição e aumenta a complexidade do problema em instâncias maiores.

Neste relatório verificaremos as diferenças de desempenho a partir dos tempos de execução e do speedup da versão CUDA em relação à versão em C.

2 Metodologia e Testes Realizados

2.1 Implementações e variações

Foram desenvolvidas três implementações, além da disponibilizada no enunciado (`heat.c`):

1. `heat_cuda.cu`: versão paralelizada em CUDA de `heat.c`, que recebe como parâmetros o tamanho do grid, o número de iterações e o tamanho do bloco de threads.
2. `heat_body.c`: implementação que aborda o problema sequencialmente assim como em `heat.c`, mas adiciona um corpo quente de temperatura constante no centro da sala, de dimensões 2×2 .
3. `heat_body_cuda.cu`: versão paralelizada em CUDA do código anterior, que deve produzir os mesmos resultados e recebe como parâmetros os mesmos de `heat_cuda.cu`

Cada uma dessas versões foi executada com variações no tamanho do bloco de threads 8×8 e 16×16) e no tamanho do grid, com os valores de n sendo 512, 1024, 2048, 4096 e 8192, totalizando 10 combinações. Os tempos foram comparados com as versões sequenciais para cada respectivo n . Cada combinação foi executada 10 vezes para calcular a média e o intervalo de confiança.

2.2 Objetivos dos Testes

Os principais objetivos dos testes eram:

1. Avaliar os impactos da paralelização via CUDA no tempo de execução.
2. Avaliar os impactos do tamanho do grid no tempo de execução.
3. Avaliar os impactos do tamanho do bloco de threads no tempo de execução.
4. Garantir que a saída do código paralelizado era equivalente à do código sequencial.

2.3 Procedimento Experimental

Os testes foram realizados em uma máquina multi-core, com processador Ryzen 5 3600 de 6 núcleos e 12 threads e 32 GB de memória, no sistema operacional Ubuntu 22.04.1 LTS, executado no Windows 10 Pro através de virtualização usando WSL. A GPU utilizada foi uma RTX 3060Ti com 4864 CUDA Cores e 8GB de VRAM. Os compiladores foram o gcc e o nvcc, para os códigos C e CUDA, respectivamente. Os testes foram executados através de programas python que executavam os códigos e registravam os tempos de execução em um arquivo csv. Cada uma das implementações gerava um arquivo texto, que poderia ser convertido em imagem pelo programa disponibilizado no enunciado e gerar saídas como a das figuras a seguir.

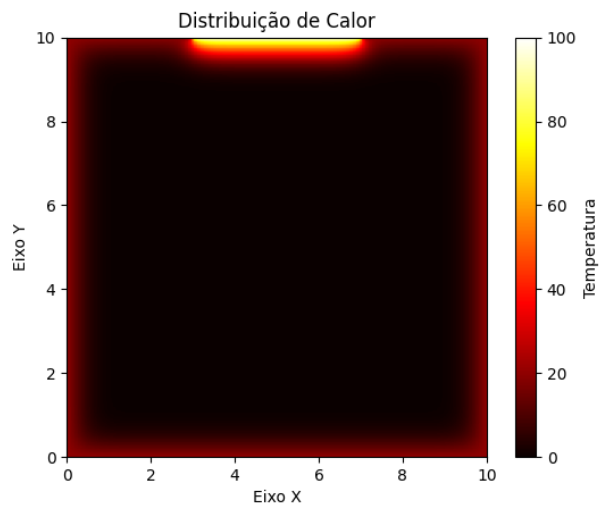


Figura 1: Exemplo de imagem gerada a partir de `heat.c` e `heat_cuda.cu`

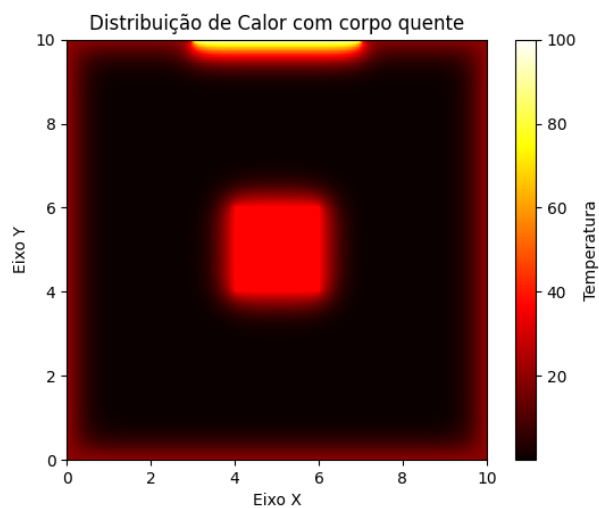


Figura 2: Exemplo de imagem gerada a partir de `heat_body.c` e `heat_body_cuda.cu`

3 Resultados

Os resultados dos testes foram organizados em duas categorias principais, com e sem corpo quente. As comparações de tempo de execução e speedup estão representadas nas figuras 3 e 5, respectivamente.

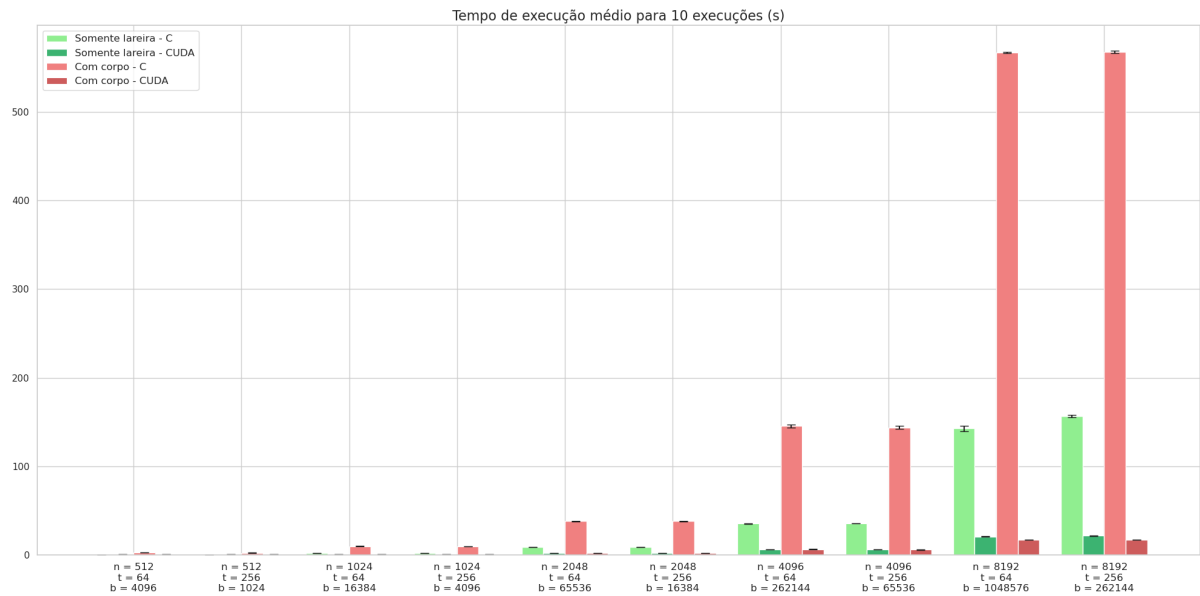


Figura 3: Comparação de Tempo de Execução entre Implementação Sequencial e Paralela

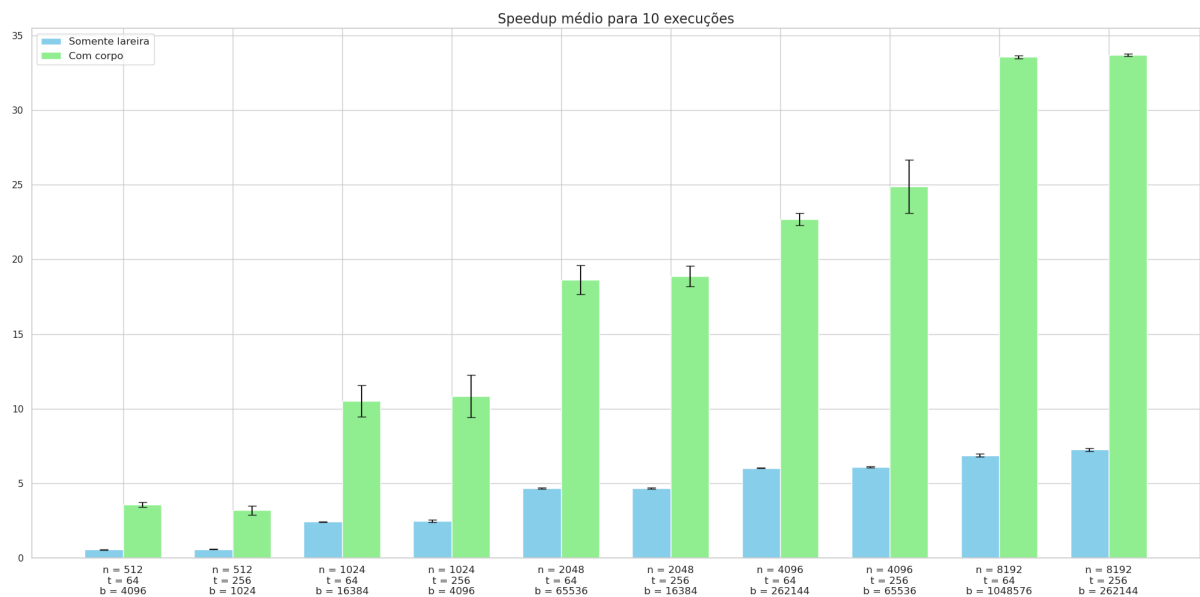


Figura 4: Comparação de Speedup Obtido pela Implementação Paralela

3.1 Impacto do Tamanho da Grade (n)

Observa-se que o tamanho da grade n tem um impacto significativo no desempenho e no speedup das implementações. Para grades menores, na categoria sem corpo quente, o speedup obtido chegou a ser menor que 1, indicando que a implementação paralela com CUDA foi mais lenta que a sequencial. Isso provavelmente se deve à sobrecarga inicial associada à configuração e lançamento dos kernels CUDA, que não é compensada pela quantidade baixa de operações em valores de n pequenos.

À medida que o tamanho da grade aumenta, o speedup tende a aumentar. Para $n = 8192$, por exemplo, o speedup alcança valores superiores a 7x na versão sem corpo quente e a 30x na versão com corpo, justificando melhor o uso da paralelização.

3.2 Efeito do Tamanho do Bloco

A variação do tamanho dos blocos de threads também influencia o desempenho. Comparando blocos de tamanho 8x8 e 16x16, observamos que blocos maiores proporcionaram um speedup maior no geral, especialmente em grades maiores. Blocos maiores permitem uma melhor utilização da memória compartilhada e reduzem a quantidade de overhead associado à criação de múltiplos blocos menores, resultando em uma maior eficiência computacional.

3.3 Impacto da Inserção do Corpo Quente

A inserção de um corpo quente na grade teve um impacto considerável no speedup obtido. Na categoria com corpo quente, mesmo para grades menores $n = 512$, o speedup ultrapassa 3x, indicando que a carga adicional imposta pelo corpo quente faz com que o uso de GPU se justifique mesmo em instâncias menores. Para grades maiores, o speedup atinge valores superiores a 30x para $n = 8192$. Um dos possíveis motivos adicionais para isso é que, em instâncias grandes, a temperatura rapidamente caía para zero em todos os pontos, o que aumentava a velocidade dos cálculos. No entanto, na versão com corpo com temperatura constante, a temperatura ao redor do corpo nunca chegaria a zero, de modo que o programa precisava continuar calculando aquelas posições em todas as iterações. Essa diferença é clara ao comparar os tempos de execução de `heat.c` e `heat.body.c`, em que `heat.body.c` foi no geral três vezes mais lento. Na figura abaixo, vemos a saída de `heat.c` com $n = 8192$.

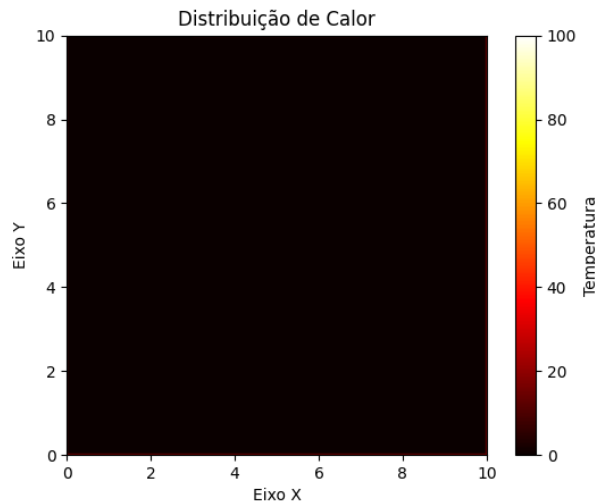


Figura 5: Imagem gerada a partir de `heat.c` com $n = 8192$

4 Conclusão

O objetivo principal deste trabalho foi analisar e comparar o desempenho das implementações sequencial e paralela (CUDA) para o problema de distribuição de calor em uma grade bidimensional, com e sem um corpo quente. Os testes realizados variaram os parâmetros de tamanho da grade e tamanho dos blocos de threads, permitindo uma avaliação abrangente das condições que influenciam o ganho de desempenho proporcionado pela paralelização em GPU.

Os resultados demonstraram que, na ausência de um corpo quente, a implementação paralela com CUDA frequentemente apresentou um speedup inferior a 1 para grades menores ($n = 512$), indicando que a sobrecarga associada ao uso da GPU não foi compensada pelo menor volume de cálculos. No entanto, à medida que o tamanho da grade aumentou ($n = 1024, 2048, 4096, 8192$), observou-se um incremento significativo no speedup, alcançando até 7 vezes para $n = 8192$ com blocos de 16×16 threads na versão sem corpo quente e 30 vezes para a versão com corpo quente. Este comportamento era esperado, já que cargas de trabalho maiores aproveitam melhor o paralelismo oferecido pelas GPUs.

A variação do tamanho dos blocos de threads mostrou-se crucial para o desempenho. Blocos maiores (16×16) proporcionaram speedups mais elevados e consistentes em comparação com blocos menores (8×8), especialmente para grades maiores. Isso se deve à melhor utilização da memória compartilhada e à redução do overhead de gerenciamento de múltiplos blocos menores, resultando em uma execução mais eficiente das operações paralelas.

A inserção de um corpo quente na grade teve um impacto considerável no speedup obtido. Com o corpo quente, mesmo para grades menores ($n = 512$), o speedup ultrapassou 3 vezes, e para grades maiores, alcançou até 30 vezes.

Além disso, os gráficos de comparação de tempo de execução e speedup ilustraram que o speedup tende a aumentar proporcionalmente ao tamanho da grade e à complexidade do problema. A implementação CUDA mostrou-se altamente eficaz para grades de grande porte e em cenários com maior complexidade, enquanto para grades menores e problemas menos complexos, a implementação sequencial ainda pode ser válida devido à sobrecarga inerente ao paralelismo.

Em resumo, este trabalho evidencia que a implementação paralela utilizando CUDA oferece vantagens significativas em termos de speedup para o problema de distribuição de calor, especialmente em cenários com grandes volumes de dados e maior complexidade. A escolha adequada do tamanho dos blocos de threads é muito relevante para otimizar o desempenho, com blocos maiores demonstrando melhor eficiência, embora seja necessário avaliar outros tamanhos para confirmar isso, já que no EP anterior verificamos que exagerar na quantidade de threads pode prejudicar o ganho de eficiência. Já a inserção de um corpo quente na grade não foi só uma coisa extra na sala, mas também um potencializador dos ganhos de performance da implementação paralela.